INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ESCOM

Trabajo Terminal

"PROTOTIPO DE UN SISTEMA WEB DE ANÁLISIS CONDUCTUAL ASISTIDO POR IA EN UN MODELO DE CONDUCTA DEPRESIVA EN RATA MEDIANTE NADO FORZADO"

2026-B066

Presentan

Frausto Robles Angel Ali

Rodriguez Verdin Sandoval Vanesa

Directores

Cordero Lopez Martha Rosa

Salas Ramírez Israel

Mayo 2026

Resumen

El presente proyecto propone el diseño, desarrollo y validación de un prototipo de sistema web asistido por inteligencia artificial orientado al análisis automatizado de la prueba de nado forzado (Forced Swim Test, FST) en ratas. El FST es un paradigma experimental estándar para la evaluación preclínica del potencial antidepresivo de moléculas en neurociencia conductual; sin embargo, su análisis actualmente depende de la observación manual cuadro a cuadro (frame a frame) de los videos, lo que implica un consumo elevado de tiempo y una variabilidad importante entre evaluadores.

El sistema propuesto integra un módulo de visión por computadora y aprendizaje supervisado capaz de clasificar cuadro a cuadro (frame a frame) las conductas de interés (nado activo, inmovilidad e intentos de escape) directamente sobre los videos proporcionados por el laboratorio colaborador, y una plataforma web que permite al investigador cargar los videos, ejecutar el análisis automático y obtener reportes cuantitativos por individuo y por sesión. Los resultados del sistema se compararán con anotaciones expertas para estimar su nivel de concordancia y la reducción de tiempo en el análisis.

El proyecto se desarrolla en colaboración con el grupo de investigación en neurociencia conductual de la Escuela Superior de Medicina y Homeopatía del IPN, que aporta el acervo de videos experimentales y el estándar de referencia del experto (gold standard) para la validación del sistema.

Palabras clave: visión por computadora, aprendizaje automático, prueba de nado forzado, farmacología conductual, sistema web, análisis conductual animal.

Índice general

Resumen I

1. Introducción 1

1.1. Antecedentes . . . 1
1.2. Planteamiento del problema . . . 2
1.3. Propuesta de solución . . . 2
1.4. Objetivo general . . . 3
1.4.1. Objetivos específicos . . . 3
1.5. Justificación . . . 4
1.6. Alcance del proyecto . . . 4
1.7. Metodología . . . 5

2. Estado del arte 7

2.1. Herramientas comerciales de análisis conductual por video . . . 7
2.1.1. EthoVision XT . . . 7
2.1.2. ANY-maze . . . 8
2.2. Herramientas de código abierto para análisis del FST . . . 8
2.2.1. ezTrack . . . 8
2.2.2. DBscorer . . . 8
2.3. Enfoques de aprendizaje profundo para el FST . . . 9
2.3.1. Red dual sin umbral (DSAAN) . . . 9
2.3.2. Red convolucional 3D-RCNN (Della Valle et al., 2025) . . . 9
2.4. Estimación de pose y seguimiento multianimales . . . 9
2.4.1. DeepLabCut . . . 9
2.4.2. YOLO-Behaviour . . . 10
2.4.3. YOLOv8 para detección en investigación conductual . . . 10

2.5. Limitaciones comunes y contexto del proyecto 10
2.6. Tabla comparativa 11

3. Marco teórico **12**
3.1. El modelo de nado forzado como paradigma preclínico 12
3.1.1. Depresión y modelos animales 12
3.1.2. La prueba de nado forzado (FST) 13
3.1.3. Conductas que se observan y cómo se interpretan 13
3.1.4. Protocolo del laboratorio colaborador 13
3.2. Redes neuronales y aprendizaje profundo 14
3.2.1. Idea general de una red neuronal 14
3.2.2. Redes convolucionales (CNN) 14
3.2.3. Transfer learning 15
3.2.4. ResNet 15
3.2.5. Clasificación de las tres conductas 15
3.3. Seguimiento de objetos en video 16
3.3.1. Trackers de correlación: CSRT y KCF 16
3.3.2. Estrategia de seguimiento en cascada 16
3.3.3. Métricas de evaluación 16
3.4. Arquitectura del sistema web 16
3.4.1. Arquitectura cliente-servidor y API REST 16
3.4.2. Procesamiento asíncrono con colas de tareas 16
3.4.3. Contenedorización con Docker 16
3.4.4. Persistencia de datos con PostgreSQL 16

4. Análisis **17**
4.1. Requerimientos 17
4.1.1. Requerimientos Funcionales (RF) 17
4.1.2. Requerimientos No Funcionales (RNF) 21
4.1.3. Reglas de negocio 23
4.2. Análisis de las herramientas a utilizar 25
4.2.1. Análisis de técnicas de detección de objetos y seguimiento 25
4.2.2. Análisis de arquitecturas de aprendizaje profundo para clasificación de conductas 26
4.2.3. Análisis de herramientas de desarrollo web, infraestructura y anotación 27

4.3. Análisis de riesgos 28
4.4. Análisis de factibilidad 31
4.4.1. Factibilidad tecnológica 31
4.4.2. Factibilidad operativa 32
4.4.3. Factibilidad económica 32
4.4.4. Factibilidad legal 33
4.5. Análisis de sostenibilidad 34
4.5.1. Sostenibilidad ambiental 34
4.5.2. Sostenibilidad social e institucional 34
4.5.3. Mantenibilidad del sistema 34

5. Desarrollo **35**
5.1. Entorno de desarrollo 35
5.2. Implementación del módulo de seguimiento (tracker.py) 35
5.2.1. Representación de imágenes y video 35
5.2.2. Modelo de fondo y sustracción 35
5.2.3. Umbralización y limpieza morfológica 36
5.2.4. Detección de contornos y selección de candidatos 36
5.2.5. Estimación de la línea de agua 36
5.3. Implementación del clasificador de conducta 37
5.4. Implementación del backend (Flask) 37
5.5. Implementación del frontend (React + Vite) 37
5.6. Integración y despliegue con Docker Compose 37

Índice de figuras

Índice de cuadros

2.1. Comparativa de herramientas y trabajos relacionados con el análisis del FST. 11

4.1. Requerimientos Funcionales (RF) [22], [23]. 18
4.2. Requerimientos No Funcionales (RNF). 21
4.3. Reglas de negocio. 24
4.4. Análisis de técnicas de detección de objetos y seguimiento. 25
4.5. Análisis de arquitecturas de aprendizaje profundo para clasificación de conductas. 26
4.6. Análisis de herramientas de desarrollo web, infraestructura y anotación. . 27
4.7. Matriz de riesgos del proyecto. 29
4.8. Estado de factibilidad por componente tecnológico. 31
4.9. Análisis de costos del proyecto. 32

Capítulo 1

Introducción

1.1. Antecedentes

La depresión es uno de los trastornos más comunes a nivel mundial. Según la Organización Mundial de la Salud, afecta a más de 280 millones de personas y es una de las principales causas de discapacidad [1]. Esto la convierte en una prioridad para la investigación farmacológica, especialmente en el desarrollo de nuevos tratamientos.

Para poder probar si un nuevo medicamento tiene efecto antidepresivo, los investigadores utilizan modelos animales antes de llegar a ensayos en humanos. Uno de los más usados en roedores es la prueba de nado forzado (Forced Swim Test, FST), propuesta originalmente por Porsolt et al. en 1977 [2]. En términos simples, consiste en meter a la rata en un cilindro con agua del que no puede salir, y registrar cuánto tiempo se queda quieta versus cuánto tiempo intenta escapar. Los antidepresivos reducen el tiempo que el animal pasa inmóvil [3], [4].

El problema es que analizar esos videos es todavía un proceso completamente manual: alguien tiene que ver la grabación y cronometrar cada conducta para cada animal. Eso toma mucho tiempo y depende del criterio de quien lo hace, lo que puede dar resultados distintos entre diferentes evaluadores [4].

En los últimos años han surgido herramientas como EthoVision XT [5] o ezTrack [6] que permiten automatizar parte del proceso. Sin embargo, tienen limitaciones importantes: algunas son demasiado costosas, otras solo detectan si el animal se mueve o no sin distinguir el tipo de conducta, y en general están pensadas para condiciones de grabación más controladas que las que se tienen en el laboratorio.

El Laboratorio de Neurociencia Conductual de la ENMyH-IPN, dirigido por el Dr. Is-

rael Salas Ramírez, trabaja con este paradigma de forma regular y cuenta con un acervo de videos que actualmente se analizan de manera manual. A partir de esa necesidad concreta surge este Trabajo Terminal.

1.2. Planteamiento del problema

El laboratorio colaborador realiza experimentos de nado forzado periódicamente, evaluando grupos de ratas para medir el efecto de distintas moléculas. Cada experimento genera videos donde se graban cuatro animales simultáneamente desde arriba, con teléfono o tableta, en dos sesiones: 20 minutos el primer día y 5 minutos el segundo día.

Hoy en día, el análisis de esos videos se hace completamente a mano: un investigador reproduce el video y va anotando con un cronómetro cuándo la rata nada, cuándo está inmóvil y cuándo intenta escalar. Esto implica varios problemas:

- **Tiempo:** analizar manualmente un experimento completo puede llevar varias horas, dependiendo del número de animales y sesiones.
- **Subjetividad:** dos personas pueden tener criterios ligeramente distintos para decidir cuándo empieza o termina una conducta, lo que afecta la reproducibilidad.
- **Sin registro centralizado:** los resultados se guardan en archivos separados (hojas de cálculo, notas), sin una forma fácil de comparar sesiones o grupos.
- **Análisis temporal limitado:** hacer un desglose minuto a minuto de las conductas sería muy valioso para ver la dinámica del efecto farmacológico, pero hacerlo manualmente es prácticamente inviable.

Todo esto limita la capacidad del laboratorio para procesar más experimentos y dificulta la estandarización de sus resultados.

1.3. Propuesta de solución

La propuesta es desarrollar un prototipo de sistema web que use inteligencia artificial para analizar automáticamente los videos del FST generados por el laboratorio.

El sistema tiene dos partes principales:

- **Pipeline de análisis de video:** recibe el video, identifica a cada rata dentro de su cilindro y clasifica cuadro a cuadro qué conducta está realizando (nado, inmovilidad o escalamiento) usando un modelo de aprendizaje supervisado. La salida es el tiempo total de cada conducta por animal y, opcionalmente, el desglose por minuto.
- **Plataforma web:** interfaz desde el navegador donde el investigador puede subir los videos, ver el estado del análisis, consultar los resultados por animal y por sesión, y descargar reportes. Los datos se almacenan en una base de datos para facilitar la comparación entre experimentos.

El sistema asume que los videos de entrada tienen un formato y calidad mínimos para que el procesamiento funcione correctamente: vista superior con los cuatro cilindros visibles, contraste suficiente entre el animal y el fondo, y estabilidad de la cámara. Los videos de muestra disponibles hasta ahora presentan algunas variaciones en iluminación y contraste que podrían afectar el desempeño; se evaluará durante el desarrollo qué tan robustos son los algoritmos ante esas condiciones y se documentarán los requisitos mínimos de calidad del video.

Para validar el sistema, sus resultados se compararán con las anotaciones manuales del experto del laboratorio (gold standard), usando métricas de concordancia que se definirán en la etapa de análisis.

1.4. Objetivo general

Diseñar, desarrollar y validar un prototipo de sistema web asistido por inteligencia artificial que automatice la detección y cuantificación de las conductas de la prueba de nado forzado en ratas —nado activo, inmovilidad e intentos de escape— y que sus resultados sean comparables con la evaluación manual del experto del laboratorio.

1.4.1. Objetivos específicos

1. Revisar los trabajos y herramientas existentes para el análisis automatizado del FST, e identificar los enfoques de visión por computadora y clasificación supervisada más adecuados al problema.
2. Describir con detalle el protocolo experimental del FST que usa el laboratorio, incluyendo las conductas de interés y las características de los videos, en coordinación con el Dr. Salas Ramírez.

3. Diseñar la arquitectura del sistema, definiendo los módulos de carga de videos, pipeline de análisis, base de datos y visualización de resultados.

4. Implementar el módulo de análisis conductual con visión por computadora y un clasificador supervisado para las tres conductas del FST.

5. Validar el desempeño del sistema comparando sus predicciones contra las anotaciones del experto, con métricas de concordancia previamente definidas.

6. Medir la reducción de tiempo que aporta el sistema frente al análisis manual, y documentar bajo qué condiciones podría adaptarse a otros experimentos conductuales.

1.5. Justificación

El análisis manual del FST tiene limitaciones claras: consume mucho tiempo, depende del criterio del evaluador y no hay una forma centralizada de almacenar y comparar los resultados. Un sistema automatizado como el que se propone resolvería directamente estos problemas para el laboratorio colaborador.

Además, las herramientas comerciales existentes (como EthoVision) son costosas y no están diseñadas para el flujo de trabajo específico del laboratorio, mientras que las de código abierto disponibles no clasifican las tres conductas del FST ni cuentan con una plataforma web. El sistema propuesto cubre esas dos necesidades al mismo tiempo.

Desde el punto de vista académico, el proyecto nos permite aplicar conocimientos de visión por computadora, aprendizaje automático y desarrollo web en un problema real con impacto en investigación biomédica, que es justamente el tipo de trabajo que define el perfil de egreso de la ESCOM.

1.6. Alcance del proyecto

Incluido en el alcance

- Análisis de videos del FST del laboratorio de la ENMyH-IPN, grabados desde arriba con cuatro cilindros simultáneos, bajo condiciones de grabación suficientes para que el procesamiento sea viable.

- Clasificación de tres conductas: nado activo, inmovilidad e intentos de escape. Si el sistema no logra distinguir con precisión suficiente nado y escalamiento, ambos se reportarán juntos como movilidad.
- Métricas por animal y por sesión: tiempo total de cada conducta en los 5 minutos del día 2, y desglose por minuto como funcionalidad opcional.
- Comparación entre sesiones día 1 y día 2 para el mismo animal o grupo.
- Plataforma web con carga de videos, consulta de resultados y generación de reportes.
- Validación contra el gold standard del experto con métricas de concordancia (Kappa de Cohen) y error absoluto medio.
- Documentación técnica conforme a los lineamientos de la CATT-ESCOM.

Fuera del alcance

- Análisis en tiempo real sobre video en vivo.
- Corrección automática de videos con mala iluminación, reflejos excesivos u otros problemas de calidad de grabación.
- Cualquier intervención en los experimentos con animales; eso lo hace el laboratorio.
- Despliegue institucional definitivo; el producto entregado es un prototipo funcional.
- Análisis de paradigmas conductuales distintos al FST en esta versión del sistema.

1.7. Metodología

El proyecto se desarrolla bajo una metodología ágil basada en Scrum, adaptada a un equipo de dos personas con entregas en dos etapas académicas (TT1 y TT2). La razón principal de elegir este enfoque es que el proyecto tiene componentes muy distintos entre sí (el módulo de video, el clasificador, la web, la base de datos) y no tiene mucho sentido diseñar todo desde el principio si no sabemos cómo va a quedar el módulo de seguimiento hasta probarlo.

Los sprints duran entre 2 y 4 semanas, y al final de cada uno hay un entregable concreto que puede revisarse con el director.

Las herramientas que se usarán son: Python con Flask para el backend y el pipeline de análisis; OpenCV y TensorFlow/scikit-learn para el procesamiento de video y clasificación; PostgreSQL para la base de datos; React.js para el frontend; BORIS para anotar el gold standard; y Git con Docker para control de versiones y despliegue.

Capítulo 2

Estado del arte

Antes de definir cómo va a funcionar el sistema, revisamos qué herramientas y trabajos existen para el análisis automatizado de la prueba de nado forzado. La búsqueda se centró en tres áreas: herramientas de software para análisis conductual, métodos de seguimiento de animales con visión por computadora, y enfoques de clasificación de conducta basados en aprendizaje profundo.

2.1. Herramientas comerciales de análisis conductual por video

2.1.1. EthoVision XT

EthoVision XT (Noldus Information Technology) es probablemente el software más usado en laboratorios de neurociencia conductual para el seguimiento automático de animales en video [5]. Funciona detectando al animal en cada cuadro mediante sustracción de fondo y umbralización, y registra su posición, velocidad y tiempo de inmovilidad.

Para el FST, EthoVision puede detectar si el animal se mueve o no, pero no diferencia entre los tres tipos de conducta (nado, inmovilidad, escalamiento) que son relevantes para el análisis farmacológico [7]. Además, requiere condiciones de iluminación bastante controladas para funcionar bien, y su licencia comercial tiene un costo elevado que no es accesible para todos los laboratorios.

2.1.2. ANY-maze

ANY-maze (Stoelting Co.) es otro software comercial con uso similar al de EthoVision. Tiene una interfaz más sencilla y accesible, pero comparte las mismas limitaciones básicas: necesita condiciones visuales controladas y no clasifica las subconductas del FST por separado.

2.2. Herramientas de código abierto para análisis del FST

2.2.1. ezTrack

ezTrack [6] es una herramienta de código abierto desarrollada en Python, pensada para investigadores sin mucha experiencia en programación. Se ejecuta sobre Jupyter Notebooks y cuantifica el movimiento del animal comparando diferencias entre cuadros consecutivos.

Es útil para experimentos simples, pero solo hace clasificación binaria (se mueve/no se mueve) y asume condiciones de grabación razonablemente controladas. No está pensada para vista superior con varios cilindros al mismo tiempo.

2.2.2. DBscorer

DBscorer [8] es la herramienta de código abierto más específica para el FST y para el test de suspensión de cola (TST). Fue desarrollada en MATLAB por el Indian Institute of Science y tiene una interfaz gráfica bastante intuitiva. Cuantifica la inmovilidad comparando el tamaño de la región del animal entre cuadros consecutivos.

Representa un avance importante frente al análisis completamente manual, pero tiene limitaciones relevantes para este proyecto: es sensible a cambios de iluminación y reflejos del agua, solo distingue móvil/inmóvil sin separar nado de escalamiento, está diseñada para un solo animal a la vez desde perspectiva lateral, y no tiene interfaz web.

2.3. Enfoques de aprendizaje profundo para el FST

2.3.1. Red dual sin umbral (DSAAN)

Meng et al. [9] propusieron un método llamado DSAAN que usa dos ramas paralelas para procesar información de apariencia y de movimiento del animal por separado, y luego las combina para clasificar el estado cuadro a cuadro. Lo interesante es que alcanza 88.7 % de precisión con apenas 94 imágenes etiquetadas, lo cual es un resultado bastante bueno para el tamaño de dataset que tendríamos disponible.

El método fue validado en condiciones controladas de laboratorio; los autores no abordan el comportamiento del sistema ante variaciones de iluminación o reflejos del agua.

2.3.2. Red convolucional 3D-RCNN (Della Valle et al., 2025)

Este es el trabajo más reciente que encontramos sobre clasificación automática en el FST. Della Valle et al. [7] entrenaron un modelo basado en redes residuales 3D que procesa segmentos de video para capturar el movimiento en el tiempo. Es de los pocos sistemas que distingue las tres conductas (nado, inmovilidad y escalamiento); los propios autores señalan que la mayoría de sistemas anteriores solo pueden diferenciar entre inmovilidad y conductas activas en general.

Al igual que los demás trabajos revisados, fue validado en condiciones controladas.

2.4. Estimación de pose y seguimiento multianimales

2.4.1. DeepLabCut

DeepLabCut [10] es la herramienta estándar para seguimiento de pose en animales sin necesidad de marcadores físicos. Usa redes convolucionales para detectar puntos clave del cuerpo (nariz, patas, cola) definidos por el usuario. Su versión multianimales, maDLC [11], permite trabajar con varios individuos al mismo tiempo.

Sturman et al. [12] demostraron que un clasificador entrenado sobre la representación esquelética de DeepLabCut puede eliminar la variabilidad entre evaluadores en el FST. Sin embargo, requiere anotar manualmente puntos clave en decenas de cuadros para entrenarlo, lo cual implica un esfuerzo de etiquetado considerable.

2.4.2. YOLO-Behaviour

Chan et al. [13] presentaron un sistema que combina YOLOv8 para detectar al animal y un clasificador de conducta que actúa sobre la región recortada. La ventaja de separar la detección de la clasificación es que el clasificador trabaja sobre una imagen más limpia, sin distracciones del fondo. Ese enfoque de dos etapas es el que se retoma en este proyecto.

2.4.3. YOLOv8 para detección en investigación conductual

Stadler et al. [14] evaluaron YOLOv8 para detección de animales en estudios de comportamiento y encontraron que con entre 100 y 350 imágenes etiquetadas se puede lograr una detección casi perfecta en fondos consistentes. También encontraron que el modelo no generaliza bien cuando el fondo cambia mucho entre condiciones, lo cual es un punto a considerar en el entrenamiento del sistema.

2.5. Limitaciones comunes y contexto del proyecto

Un patrón que se repite en todos los trabajos revisados es que fueron desarrollados y validados bajo condiciones de grabación bastante controladas: iluminación uniforme, fondo contrastante, sin reflejos problemáticos. Las herramientas basadas en umbralización (DBscorer, ezTrack) son especialmente sensibles a esas condiciones. Los métodos de aprendizaje profundo son más robustos en principio, pero su rendimiento depende en gran medida de que el conjunto de entrenamiento represente las condiciones reales de uso.

En ese sentido, los videos disponibles del laboratorio colaborador presentan algunas características que pueden suponer un reto: grabación con cámara de celular, variaciones de iluminación entre sesiones, y ratas de pelaje claro con bajo contraste frente al cilindro. El sistema propuesto no garantiza robustez ante todas esas condiciones desde el inicio; parte del trabajo de desarrollo será evaluar qué tan bien funciona con los videos disponibles y definir los requisitos mínimos de calidad de entrada. La hipótesis de trabajo es que los videos del laboratorio son suficientemente buenos para que el sistema funcione, pero eso se confirmará o ajustará durante las pruebas.

2.6. Tabla comparativa

La Tabla 2.1 resume los trabajos revisados en relación con los criterios más importantes para el proyecto.

Cuadro 2.1: Comparativa de herramientas y trabajos relacionados con el análisis del FST.

Sistema	**Espec. FST**	**3 conductas**	**DL**	**Web**	**Abierto**
EthoVision XT	Parc.	No	No	No	No
ANY-maze	Parc.	No	No	No	No
ezTrack	Parc.	No	No	No	Sí
DBscorer	Sí	No	No	No	Sí
DSAAN	Sí	No	Sí	No	No
3D-RCNN	Sí	Sí	Sí	No	No
DeepLabCut	Parc.	Parc.	Sí	No	Sí
YOLO-Behaviour	No	—	Sí	No	Sí
Propuesta	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**

Ningún sistema existente cubre los cinco criterios al mismo tiempo. Los comerciales no son abiertos y no clasifican las tres conductas. Los de código abierto no usan aprendizaje profundo ni tienen interfaz web. Los de aprendizaje profundo más recientes sí clasifican las tres conductas pero no tienen plataforma web. El sistema propuesto busca cubrir los cinco, con la salvedad de que el desempeño real dependerá de la calidad de los videos con los que se trabaje.

Capítulo 3

Marco teórico

Este capítulo describe los conceptos teóricos en los que se apoya el sistema. Se organiza en tres bloques: el modelo biológico del FST y las conductas que interesa clasificar; los fundamentos de redes neuronales y aprendizaje profundo para la clasificación de conductas; y los elementos de arquitectura del sistema web. Las técnicas específicas de procesamiento de video utilizadas en el módulo de seguimiento (sustracción de fondo, umbralización, contornos, línea de agua) se describen en el capítulo 6, dentro del desarrollo del módulo correspondiente.

3.1. El modelo de nado forzado como paradigma preclínico

3.1.1. Depresión y modelos animales

La depresión mayor es uno de los trastornos más frecuentes a nivel mundial. La OMS estima que afecta a más de 280 millones de personas y es una de las principales causas de discapacidad [1]. Para estudiar nuevos tratamientos antes de probarlos en humanos, los investigadores usan modelos animales que reproducen algunos aspectos del trastorno.

Para que un modelo animal sea considerado válido, debe cumplir con tres criterios [3]: tener *validez de constructo* (los mecanismos biológicos involucrados deben ser parecidos a los del trastorno humano), *validez de cara* (los síntomas observables deben ser similares a los clínicos) y *validez predictiva* (el modelo debe responder a los fármacos que funcionan en pacientes).

3.1.2. La prueba de nado forzado (FST)

La prueba de nado forzado fue propuesta por Porsolt et al. en 1977 [2]. Consiste en poner a la rata en un cilindro lleno de agua del que no puede salir, y registrar cómo se comporta. Al principio el animal intenta escapar, pero después de un rato empieza a quedarse flotando. Esa conducta pasiva se interpreta como *desesperanza conductual* (en inglés *behavioral despair*), que se asocia con la pérdida de motivación propia de la depresión [4].

La prueba funciona bien para evaluar antidepresivos porque la mayoría de los fármacos que son efectivos en humanos también reducen el tiempo de inmovilidad en ratas [15]. Es uno de los experimentos más citados en neurociencia conductual: desde el 2000 hay más de 8,000 artículos que lo reportan [7].

3.1.3. Conductas que se observan y cómo se interpretan

En el FST se pueden distinguir tres tipos de conducta [4], [15]. Son mutuamente excluyentes, o sea que en cada momento la rata solo está haciendo una:

- **Inmovilidad**: el animal flota haciendo solo lo necesario para no hundirse. Es el indicador principal de desesperanza conductual.
- **Nado activo**: movimientos de paladas que lo desplazan horizontalmente. Está relacionado con el sistema serotoninérgico; los antidepresivos tipo ISRS (como la fluoxetina) aumentan esta conducta.
- **Escalamiento**: movimientos rápidos hacia las paredes del cilindro, con las patas delanteras tocando o rompiendo la superficie del agua. Se asocia al sistema noradrenérgico; la desipramina, por ejemplo, la incrementa.

Distinguir estas tres conductas es importante porque permite saber sobre qué sistema neurotransmisor está actuando el fármaco evaluado. Un sistema que solo detecta si la rata se mueve o no pierde esa información.

3.1.4. Protocolo del laboratorio colaborador

En el laboratorio de la ENMyH, las ratas se meten individualmente en cilindros transparentes con agua. Hay dos sesiones: 20 minutos el día 1 (línea base) y 5 minutos el día 2

(después del tratamiento), con 24 horas de diferencia. Los videos se graban desde arriba con cámara de celular, capturando los cuatro cilindros al mismo tiempo. La iluminación y el nivel del agua varían entre experimentos, y con ratas de pelaje claro el contraste contra el cilindro puede ser bastante bajo. Estas condiciones representan el caso de uso principal del sistema y determinarán en qué medida los algoritmos funcionan correctamente; los requisitos mínimos de calidad del video se definirán durante las pruebas de validación.

3.2. Redes neuronales y aprendizaje profundo

3.2.1. Idea general de una red neuronal

Una red neuronal es un modelo que aprende a partir de ejemplos. Está formada por capas de "neuronas" (en realidad son solo funciones matemáticas simples) que se conectan entre sí. Cada neurona recibe varios valores, los pondera con unos pesos, los suma y aplica una función de activación. La función más usada hoy en día es ReLU, que devuelve el valor si es positivo y cero si es negativo.

Los pesos no se definen a mano: se aprenden durante el entrenamiento. El proceso es iterativo: se le da al modelo un ejemplo con su respuesta correcta, se ve cuánto se equivocó, y se ajustan un poquito los pesos para que la próxima vez lo haga mejor. Eso se hace con el algoritmo de retropropagación y un optimizador como Adam [16], que es una versión mejorada del descenso por gradiente clásico.

3.2.2. Redes convolucionales (CNN)

Las CNN son redes neuronales diseñadas específicamente para trabajar con imágenes [17]. La idea principal es que en vez de conectar cada píxel con cada neurona (lo que sería inmanejable), se usan filtros pequeños que se deslizan por toda la imagen. El mismo filtro se aplica en cada posición, lo cual reduce mucho la cantidad de parámetros y hace que la red pueda detectar un patrón sin importar dónde esté en la imagen.

Las capas del principio aprenden a detectar cosas simples como bordes o cambios de color. Las capas más profundas van combinando esas características básicas para detectar formas más complejas, y al final hay capas completamente conectadas que hacen la clasificación.

3.2.3. Transfer learning

Entrenar una red desde cero requiere muchos datos etiquetados, generalmente cientos de miles de imágenes. Para este proyecto eso no es viable porque el laboratorio tiene un número limitado de videos.

La solución es el *transfer learning* [18]: se usa una red que ya fue entrenada en otro dataset grande (en este caso ImageNet, con más de un millón de imágenes) y se adapta para la tarea nueva. En la práctica se reemplaza la última capa del modelo (que antes clasificaba 1,000 objetos diferentes) por una capa nueva con solo 3 salidas (una por conducta), y se re-entrena solo esa parte final con los videos del laboratorio. Las capas anteriores ya saben detectar bordes, texturas y formas generales, y eso sí se puede aprovechar.

3.2.4. ResNet

ResNet [19] es una arquitectura CNN muy usada como punto de partida para transfer learning. La novedad que introdujo fue agregar conexiones directas que se "saltan" una o más capas (por eso se llaman *skip connections* o conexiones residuales). Eso facilita mucho el entrenamiento cuando la red es muy profunda, porque el error puede fluir más fácilmente hacia atrás sin desvanecerse.

Para este proyecto se considera usar ResNet-18 o ResNet-50 como base del clasificador, dependiendo de los resultados de las pruebas iniciales.

3.2.5. Clasificación de las tres conductas

El clasificador recibe como entrada el recorte de la imagen alrededor de la rata (el bounding box), normalizado a un tamaño fijo. La capa de salida produce tres valores de probabilidad (uno por conducta) y se predice la que tenga mayor valor. Para entrenar el modelo se necesitan cuadros de video etiquetados por el Dr. Salas Ramírez, indicando qué conducta realiza cada rata. Esas etiquetas se obtienen con la herramienta BORIS y sirven también como gold standard para la validación.

3.3. Seguimiento de objetos en video

3.3.1. Trackers de correlación: CSRT y KCF

Los trackers de correlación aprenden a reconocer cómo se ve el objeto que siguen y en cada cuadro buscan la región de la imagen que más se le parece. CSRT [20] es más preciso pero tarda más en procesar cada cuadro. KCF [21] es más rápido pero menos robusto ante cambios bruscos de apariencia. En el sistema se usa CSRT como primera opción, con KCF como respaldo si CSRT no está disponible en la versión de OpenCV instalada.

3.3.2. Estrategia de seguimiento en cascada

Para que ningún cuadro quede sin detección, el módulo implementa tres niveles de respaldo. Primero intenta detectar a la rata por segmentación (sustracción de fondo). Si eso falla o el resultado no es consistente con el cuadro anterior, usa la predicción del tracker CSRT. Y si ese también falla, congela la última posición conocida, asignándole una confianza que va disminuyendo con el tiempo hasta un mínimo.

3.3.3. Métricas de evaluación

Para validar el sistema se compararán sus predicciones contra las anotaciones manuales del experto. La métrica principal es la kappa de Cohen (κ), que mide la concordancia entre dos evaluadores descontando lo que ocurriría por pura casualidad. Se busca alcanzar $\kappa > 0{,}80$, que corresponde a concordancia casi perfecta según la escala de Landis y Koch. También se calcula el error absoluto medio en segundos para tener una idea del error acumulado por animal.

3.4. Arquitectura del sistema web

3.4.1. Arquitectura cliente-servidor y API REST

3.4.2. Procesamiento asíncrono con colas de tareas

3.4.3. Contenedorización con Docker

3.4.4. Persistencia de datos con PostgreSQL

Capítulo 4

Análisis

En este capítulo se definen los requerimientos del sistema, las reglas que deben respetarse durante su operación, los riesgos que podrían afectar el desarrollo y la factibilidad del proyecto desde cuatro ángulos: tecnológico, operativo, económico y legal. Todo lo que se describe aquí sirve como punto de partida para el diseño e implementación que se presentan en el Capítulo 5.

4.1. Requerimientos

Para levantar los requerimientos se realizaron reuniones con el Dr. Israel Salas Ramírez, responsable del laboratorio de neurociencia conductual de la ENMyH-IPN, y se analizó paso a paso cómo trabaja actualmente el laboratorio con los videos del FST. De esas sesiones salieron dos tipos de requerimientos: los funcionales, que dicen qué debe hacer el sistema, y los no funcionales, que dicen bajo qué condiciones debe hacerlo [22], [23].

4.1.1. Requerimientos Funcionales (RF)

Los requerimientos funcionales están organizados por módulo y tienen tres niveles de prioridad: **Alta** (sin esto el prototipo no sirve), **Media** (importante pero puede quedar para una iteración posterior) y **Baja** (sería bueno tenerlo pero no es urgente).

Cuadro 4.1: Requerimientos Funcionales (RF) [22], [23].

ID	Módulo	Descripción	Prioridad	Actor
RF-01	Autenticación	El sistema debe permitir iniciar sesión con correo electrónico y contraseña.	Alta	Investigador / Admin
RF-02	Autenticación	El sistema debe cerrar la sesión del usuario de forma segura, invalidando el token activo.	Alta	Investigador / Admin
RF-03	Autenticación	El sistema debe permitir recuperar la contraseña enviando un enlace al correo registrado.	Media	Investigador
RF-04	Autenticación	El sistema debe manejar dos roles con permisos distintos: Investigador y Administrador.	Alta	Admin
RF-05	Gestión de usuarios	El Administrador debe poder crear, modificar y desactivar cuentas de usuario.	Alta	Admin
RF-06	Gestión de usuarios	El Investigador debe poder actualizar su nombre, correo y contraseña desde su perfil.	Media	Investigador
RF-07	Gestión de usuarios	**PENDIENTE — PREGUNTAR AL DR. SALAS RAMÍREZ:** ¿los investigadores se registran solos (con confirmación por correo institucional) o solo el Administrador puede crear cuentas? Se define antes del Sprint 3.	Alta	Admin
RF-08	Carga de video	El sistema debe aceptar videos en formato MP4. AVI y MOV quedan como alternativa cuando esté disponible la conversión automática.	Alta	Investigador

ID	Módulo	Descripción	Prioridad	Actor
RF-09	Carga de video	El sistema debe permitir subir uno o dos videos por experimento: el del Día 1 (20 minutos) y el del Día 2 (5 minutos) del protocolo FST.	Alta	Investigador
RF-10	Carga de video	El sistema debe verificar que el archivo subido sea un video válido antes de procesarlo; si no lo es, debe rechazarlo con un mensaje claro.	Alta	Sistema
RF-11	Carga de video	Al subir el video, el investigador debe poder registrar los metadatos del experimento: nombre, fecha, tratamiento o condición, y número esperado de animales.	Alta	Investigador
RF-12	Análisis conductual	El análisis debe iniciarse automáticamente en cuanto termine la carga y se valide el video, sin que el investigador tenga que hacer nada más.	Alta	Sistema
RF-13	Análisis conductual	El sistema debe detectar solo, sin ayuda del usuario, la posición y número de cilindros que aparecen en el video.	Alta	Sistema
RF-14	Análisis conductual	El sistema debe clasificar cuadro a cuadro la conducta de cada animal: nado activo, inmovilidad o intentos de escape [4].	Alta	Sistema
RF-15	Análisis conductual	Mientras el análisis corre, el investigador debe ver el progreso en tiempo real con una barra o indicador de porcentaje.	Alta	Investigador
RF-16	Análisis conductual	Si el análisis falla, el sistema debe avisarle al usuario qué salió mal y darle la opción de descargar un reporte de diagnóstico en PDF.	Alta	Investigador

ID	Módulo	Descripción	Prioridad	Actor
RF-17	Resultados	El sistema debe mostrar el tiempo total en segundos de cada conducta, separado por animal y por sesión.	Alta	Investigador
RF-18	Resultados	El sistema debe desglosar la distribución de conductas por minuto para cada animal.	Media	Investigador
RF-19	Resultados	Cuando los dos videos del mismo experimento estén procesados, el sistema debe mostrar una comparación entre el Día 1 y el Día 2.	Alta	Investigador
RF-20	Resultados	El investigador debe poder descargar el reporte de resultados en PDF con un botón dedicado en la interfaz.	Alta	Investigador
RF-21	Resultados	El investigador debe poder descargar los datos de resultados en CSV con un botón dedicado en la interfaz.	Alta	Investigador
RF-22	Gestión de experimentos	El sistema debe mostrar la lista de experimentos del investigador activo con su estado (en proceso, completado, con error) y fecha de creación.	Alta	Investigador
RF-23	Gestión de experimentos	El investigador debe poder consultar los resultados de cualquier experimento que ya haya sido procesado.	Alta	Investigador
RF-24	Gestión de experimentos	El sistema debe borrar automáticamente los archivos de video 30 días después de que el análisis termine bien, notificándole al usuario con un aviso en la interfaz.	Media	Sistema
RF-25	Gestión de experimentos	Los resultados y reportes deben conservarse indefinidamente, aunque el video original ya haya sido borrado.	Alta	Sistema

ID	Módulo	Descripción	Prioridad	Actor
RF-26	Gestión de experimentos	El investigador debe poder borrar un experimento de su historial; la acción es permanente (incluye videos y resultados) y requiere confirmación explícita.	Media	Investigador
RF-27	Administración	El Administrador debe poder ver la lista de usuarios registrados y su actividad reciente.	Media	Admin
RF-28	Administración	El Administrador debe poder revisar el estado general del sistema: espacio en disco y tamaño de la cola de procesamiento.	Baja	Admin

4.1.2. Requerimientos No Funcionales (RNF)

Los requerimientos no funcionales dicen cómo debe comportarse el sistema más allá de las funciones que realiza: qué tan rápido debe responder, qué tan seguro tiene que ser, desde dónde se puede usar y cuánto esfuerzo cuesta mantenerlo.

Cuadro 4.2: Requerimientos No Funcionales (RNF).

ID	Categoría	Descripción	Métrica / Criterio de aceptación
RNF-01	Rendimiento	El análisis de un video de 5 minutos (Día 2 del FST, resolución típica de celular) debe terminar en un tiempo razonable para el flujo de trabajo del laboratorio. **PENDIENTE — PREGUNTAR AL DR. SALAS RAMÍREZ:** ¿cuánto tiempo de espera es aceptable? Se mide y define durante el Sprint 6.	Tiempo máximo a medir y documentar en el Sprint 6.

ID	Categoría	Descripción	Métrica / Criterio de aceptación
RNF-02	Rendimiento	La interfaz web debe responder a las acciones del usuario en menos de 3 segundos en condiciones normales de red.	Tiempo de respuesta $\leq$ 3 s en el 95 % de las solicitudes.
RNF-03	Disponibilidad	El sistema debe estar disponible en el horario de trabajo del laboratorio. No se requiere disponibilidad 24/7 para el prototipo.	95 % de disponibilidad de lunes a viernes, 8:00–20:00 h.
RNF-04	Seguridad	Solo los usuarios autenticados pueden acceder al sistema. Los videos y resultados de un usuario no deben ser visibles para otros sin autorización explícita.	Autenticación con JWT. Datos aislados por usuario en la base de datos.
RNF-05	Seguridad	Las contraseñas deben guardarse cifradas, nunca en texto plano.	Hash con sal usando bcrypt o equivalente.
RNF-06	Usabilidad	Un investigador sin experiencia en programación debe poder subir un video, esperar el análisis y descargar el reporte sin necesitar ayuda técnica.	Prueba con al menos un usuario del laboratorio durante el Sprint 6.
RNF-07	Compatibilidad	El sistema debe funcionar desde Chrome, Firefox y Edge actuales, sin instalar nada en el equipo del usuario.	Pruebas en los tres navegadores en sus versiones actuales.
RNF-08	Almacenamiento	Los videos se guardan 30 días después de que el análisis termina bien y luego se borran automáticamente. Los resultados y reportes se conservan para siempre.	Tarea programada (*cron job*) de limpieza que corre cada día.

ID	Categoría	Descripción	Métrica / Criterio de aceptación
RNF-09	Portabilidad	El sistema debe poder levantarse con Docker Compose en cualquier equipo que tenga Docker instalado, sin dependencias extra en el sistema anfitrión.	Despliegue documentado en al menos dos entornos distintos.
RNF-10	Escalabilidad	Debe ser posible agregar soporte para otros paradigmas conductuales reconfigurando solo el módulo de clasificación, sin tocar la plataforma web.	Diseño modular verificado en la revisión de arquitectura del Sprint 2.
RNF-11	Mantenibilidad	El código debe estar documentado y separado en módulos independientes (backend, pipeline de IA, frontend), siguiendo PEP 8 en Python y ESLint en JavaScript.	Revisión de calidad de código en cada cierre de sprint.
RNF-12	Concurrencia	**PENDIENTE — PREGUNTAR AL DR. SALAS RAMÍREZ:** ¿cuántos usuarios y análisis simultáneos necesita soportar el sistema, y con qué infraestructura cuenta el laboratorio? Se define antes del Sprint 6 (pruebas de carga).	Pendiente. Se documenta antes del Sprint 6.

4.1.3. Reglas de negocio

Las reglas de negocio son los límites que el sistema siempre tiene que respetar, independientemente de cómo se implemente. Algunas vienen del protocolo experimental del FST, otras del sentido común de la ingeniería de software y otras del contexto institucional del laboratorio [22]. Están redactadas de forma que cualquiera pueda verificar si el sistema las cumple o no.

Cuadro 4.3: Reglas de negocio.

ID	Regla	Justificación
RN-01	Un video pertenece a exactamente un experimento. No se puede reutilizar un video que ya se subió para asociarlo a un experimento distinto.	Garantiza que los resultados siempre sean trazables a su origen.
RN-02	Un experimento tiene como máximo dos videos: uno del Día 1 (20 minutos) y uno del Día 2 (5 minutos).	Así funciona el protocolo FST del laboratorio colaborador [3].
RN-03	La comparación entre Día 1 y Día 2 solo se genera si los dos videos del mismo experimento ya fueron procesados exitosamente.	No tiene sentido comparar si falta la mitad de los datos.
RN-04	El análisis lo inicia el sistema solo, cuando termina la carga. El investigador no puede iniciarlo, pausarlo ni reiniciarlo a mano.	Evita que la cola de procesamiento quede en estados inconsistentes.
RN-05	Los videos se borran automáticamente 30 días después de que el análisis termina. La acción es irreversible y el usuario recibe un aviso en la interfaz antes de que ocurra.	Controla el uso del almacenamiento y protege los datos del laboratorio.
RN-06	Los resultados y reportes (PDF y CSV) se conservan aunque el video original ya haya sido borrado.	Los datos de investigación tienen valor a largo plazo.
RN-07	En cada cuadro del video, a cada animal se le asigna exactamente una conducta: nado activo, inmovilidad o escalamiento. No puede tener dos al mismo tiempo.	El FST define estas conductas como mutuamente excluyentes [4], [15].
RN-08	Cada investigador solo puede ver sus propios experimentos y resultados. El Administrador puede ver todo.	Control de acceso básico para proteger la confidencialidad de los datos.

ID	Regla	Justificación
RN-09	Borrar un experimento es permanente: se van el video, los resultados y los reportes. El sistema pide confirmación explícita antes de ejecutarlo.	Previene borrados accidentales de datos de investigación.
RN-10	Antes de procesar cualquier video, el sistema verifica que el archivo tenga un formato válido. Si no lo tiene, lo rechaza sin gastar recursos de cómputo.	No tiene sentido poner en cola un archivo que el pipeline no puede procesar.

4.2. Análisis de las herramientas a utilizar

La elección del *stack* tecnológico no fue arbitraria. Para cada componente del sistema se evaluaron las opciones disponibles considerando qué tan maduras son, si funcionan bien con el resto de las herramientas, qué tan fácil es encontrar documentación y soporte, y si el equipo ya tenía experiencia con ellas. Las herramientas se agrupan en cuatro áreas.

4.2.1. Análisis de técnicas de detección de objetos y seguimiento

Cuadro 4.4: Análisis de técnicas de detección de objetos y seguimiento.

Herramienta	Características principales	Por qué se eligió
OpenCV 4.x [24]	Biblioteca estándar para procesamiento de imágenes y video. Incluye sustracción de fondo, umbralización, morfología, contornos y los trackers CSRT y KCF.	Es la herramienta de referencia para este tipo de trabajo en Python y es compatible con YOLOv8 y TensorFlow sin configuración extra.
Ultralytics YOLOv8 [25]	Detector de objetos estado del arte. Funciona bien con datasets pequeños ($\geq$50 imágenes) y produce *bounding boxes* precisos en fondos consistentes.	Ya se usó para entrenar el detector de rata (`weights/rat.pt`), así que la viabilidad técnica está confirmada.

Herramienta	**Características principales**	**Por qué se eligió**
Sustracción de fondo por mediana [26]	El fondo se estima como la mediana píxel a píxel de los primeros N cuadros. Funciona bien aunque la rata esté inmóvil al inicio del video.	Más simple y predecible que el modelo de mezcla gaussiana [27] para el caso de cámara fija que tenemos. Ya está implementado en `tracker.py`.
Tracker CSRT [20]	Tracker de correlación que mantiene el seguimiento entre cuadros. Es más preciso que KCF aunque un poco más lento.	Primera opción para el seguimiento. Si CSRT no está disponible en la versión de OpenCV instalada, se usa KCF [21] como respaldo.

4.2.2. Análisis de arquitecturas de aprendizaje profundo para clasificación de conductas

Cuadro 4.5: Análisis de arquitecturas de aprendizaje profundo para clasificación de conductas.

Herramienta / Arquitectura	**Características principales**	**Por qué se eligió**
ResNet-18 [19]	Red con 18 capas y conexiones residuales, preentrenada en ImageNet. Es ligera y entrena rápido con datasets pequeños usando *transfer learning* [18].	Es la primera opción para el clasificador de conducta que opera sobre los recortes de cada rata.
ResNet-50 [19]	Versión más profunda (50 capas). Tiene más capacidad que ResNet-18 pero también cuesta más en tiempo de entrenamiento e inferencia.	Se usará como alternativa si ResNet-18 no alcanza $\kappa > 0{,}80$.

Herramienta / Arquitectura	**Características principales**	**Por qué se eligió**
TensorFlow 2.x / Keras [28]	Framework de aprendizaje profundo con una API de alto nivel que simplifica el *fine-tuning* de modelos preentrenados.	Ya se usa en el proyecto y es compatible con Docker para el despliegue.
scikit-learn [29]	Biblioteca de ML para preprocesamiento, métricas y validación cruzada.	Se necesita para calcular el coeficiente κ de Cohen y las métricas de clasificación por clase.

4.2.3. Análisis de herramientas de desarrollo web, infraestructura y anotación

Cuadro 4.6: Análisis de herramientas de desarrollo web, infraestructura y anotación.

Herramienta	**Características principales**	**Por qué se eligió**
Flask (Python) [30]	Microframework ligero para construir APIs REST. Tiene extensiones para autenticación (Flask-JWT-Extended), ORM (SQLAlchemy) y tareas asíncronas.	Es suficiente para el tamaño del prototipo y permite integrar el pipeline de IA sin necesitar un framework más pesado.
React.js + Vite	Framework frontend basado en componentes. Vite reduce mucho los tiempos de compilación durante el desarrollo.	Es la opción más adecuada para construir la barra de progreso en tiempo real y el resto de la interfaz reactiva que necesita el sistema.

Herramienta	**Características principales**	**Por qué se eligió**
PostgreSQL	Base de datos relacional con soporte para transacciones ACID y tipos de datos avanzados. Es de código abierto.	Cubre bien los patrones de acceso del sistema y se integra sin problema con SQLAlchemy.
Docker + Docker Compose [31]	Permite empaquetar todos los servicios del sistema (backend, frontend, base de datos, worker) en contenedores independientes y levantarlos juntos con un solo comando.	El laboratorio no tiene que instalar nada más allá de Docker para correr el sistema.
BORIS [32]	Software de código abierto para anotar conductas en video cuadro a cuadro. Exporta en varios formatos compatibles con análisis estadístico.	Es la herramienta con la que el Dr. Salas Ramírez va a generar el *gold standard* para validar el sistema.

4.3. Análisis de riesgos

Todo proyecto tiene cosas que pueden salir mal. Identificarlos desde el principio no significa que vayan a ocurrir, pero sí permite tener un plan antes de que sea urgente. Cada riesgo tiene una probabilidad estimada, un nivel de impacto si ocurre, y una acción concreta para reducir el daño.

Cuadro 4.7: Matriz de riesgos del proyecto.

ID	Descripción	Prob.	Impacto	Nivel	Mitigación
R-01	Los videos del laboratorio tienen reflejos, variaciones de iluminación o ratas de pelaje muy claro, y eso hace que el pipeline no funcione bien.	Alta	Alto	Alto	Definir con el laboratorio los requisitos mínimos de calidad de video y ajustar los parámetros de preprocesamiento durante las primeras pruebas.
R-02	El dataset disponible (63 imágenes) no alcanza para entrenar un clasificador de conducta que llegue a $\kappa > 0{,}80$.	Alta	Alto	Alto	Ampliar el dataset anotando más *frames* del laboratorio con BORIS. Usar *data augmentation*. Si el escalamiento sigue siendo difícil de distinguir, reportarlo junto con el nado activo como "conducta activa".
R-03	El Dr. Salas Ramírez no tiene tiempo disponible para las sesiones de validación y anotación del *gold standard*.	Media	Alto	Medio	Acordar un calendario de reuniones desde el inicio del TT2 y preparar los materiales con anticipación para aprovechar cada sesión.
R-04	Procesar un video de 5 minutos tarda más de lo que el laboratorio puede esperar en su flujo normal de trabajo.	Media	Medio	Medio	Medir dónde se van los tiempos en el pipeline y ver si reducir la resolución de análisis o agregar GPU resuelve el problema.

ID	Descripción	Prob.	Impacto	Nivel	Mitigación
R-05	Integrar el módulo de IA (Python / OpenCV) con la plataforma web (Flask / React) a través de la cola asíncrona resulta más complicado de lo esperado.	Media	Medio	Medio	Definir los contratos de API desde el Sprint 2 y hacer una prueba de integración de extremo a extremo en el Sprint 3, antes de agregar más funcionalidad.
R-06	El equipo de despliegue se queda sin espacio en disco por los videos acumulados durante los 30 días de retención.	Baja	Medio	Bajo	Documentar cuánto ocupa cada experimento. Poner alertas de disco. La eliminación automática a 30 días evita que el problema crezca indefinidamente.
R-07	El laboratorio pide cambios en los requerimientos a mitad del desarrollo: una conducta nueva o condiciones de grabación distintas.	Baja	Alto	Medio	Diseñar el clasificador de forma modular para que sea fácil agregar nuevas clases. Gestionar cualquier cambio dentro del proceso de revisión de sprint.
R-08	Desde la vista superior del video, el escalamiento y el nado activo se ven tan parecido que el modelo no puede distinguirlos con suficiente precisión.	Alta	Medio	Medio	Esto ya está contemplado en el alcance del Capítulo 1: si no se alcanza la precisión suficiente, ambas conductas se reportan juntas como “conducta activa”.

4.4. Análisis de factibilidad

El análisis de factibilidad responde a una pregunta sencilla: ¿es posible hacer este sistema? Se revisa desde cuatro ángulos para asegurarse de que no haya ningún obstáculo que haga inviable el proyecto.

4.4.1. Factibilidad tecnológica

El proyecto es tecnológicamente viable. Todas las herramientas del *stack* son de código abierto, están bien documentadas y el equipo ya tiene experiencia con la mayoría de ellas desde cursos de la carrera en la ESCOM.

Lo más importante es que el modelo de detección de rata ya funciona: se entrenó YOLOv8 con el dataset disponible, los pesos están en `weights/rat.pt` y las métricas de entrenamiento quedaron guardadas en `runs/detect/train/`. Eso confirma que al menos la parte de detección es viable con los videos que tiene el laboratorio. Para el clasificador de conducta todavía hay que anotar más *frames* con BORIS, pero la arquitectura (ResNet-18 con *transfer learning*) es perfectamente aplicable al problema con el tamaño de dataset que se puede generar.

Si el laboratorio no tiene GPU disponible para el entrenamiento, Google Colab lo resuelve gratis con hasta 12 horas de GPU por sesión.

Cuadro 4.8: Estado de factibilidad por componente tecnológico.

Componente	**Estado**
Detección de rata (YOLOv8)	Factible — modelo entrenado con métricas disponibles en `runs/detect/train/`.
Clasificador de conducta (ResNet)	Factible — requiere ampliar el dataset de anotaciones con BORIS.
Pipeline de seguimiento (OpenCV)	Factible — implementado en `tracker.py` y documentado en el Capítulo 6.
Backend Flask + PostgreSQL	Factible — estructura base en `backend/app/`.
Frontend React	Factible — esqueleto en `frontend/src/`; faltan las vistas.

Componente	Estado
Despliegue Docker Compose	Factible — `docker-compose.yml` ya está en el repositorio.
Entrenamiento con GPU	Factible — Google Colab como opción gratuita si no hay GPU local.

4.4.2. Factibilidad operativa

El sistema no le cambia el protocolo al laboratorio. El investigador sigue grabando los videos igual que siempre; lo único diferente es que en vez de analizarlos a mano, los sube a la plataforma y espera el resultado.

Como es una aplicación web, no hay nada que instalar en el equipo del usuario. Eso reduce mucho la barrera de adopción, especialmente en un laboratorio donde no todos tienen conocimientos técnicos. La prueba de usabilidad del Sprint 6 va a confirmar si esto funciona como esperamos con al menos un usuario real del laboratorio.

Los reportes que genera el sistema salen en PDF y CSV, que son los formatos que ya usa el laboratorio para sus análisis estadísticos en hojas de cálculo, R o MATLAB, así que encajan en el flujo de trabajo existente sin ningún cambio.

4.4.3. Factibilidad económica

No hay costo de licencias porque todas las herramientas son de código abierto. El desarrollo lo hace el equipo del TT sin costo adicional, y para la infraestructura de despliegue se usa el equipo del laboratorio o las computadoras del equipo de desarrollo.

Cuadro 4.9: Análisis de costos del proyecto.

Concepto	Costo estimado
Licencias de software (Python, Flask, React, PostgreSQL, Docker, YOLOv8, OpenCV, BORIS)	$0.00 — todas son de código abierto con licencias compatibles con uso académico.
Entrenamiento del modelo (GPU)	$0.00 — Google Colab gratuito con hasta 12 h de GPU por sesión.

Concepto	**Costo estimado**
Infraestructura de despliegue	Sin costo adicional — equipo del laboratorio o computadora del equipo de TT.
Almacenamiento de videos (estimado)	∼500 MB por experimento de 5 min en MP4. La eliminación automática a 30 días evita que el disco crezca indefinidamente.
Costo total de herramientas y software	**$0.00**

El proyecto es económicamente viable. No necesita financiamiento externo y no hay ningún costo que dependa de aprobaciones institucionales para empezar.

4.4.4. Factibilidad legal

Todas las herramientas del proyecto usan licencias de código abierto que permiten el uso académico sin restricciones:

- **Python:** licencia PSF — uso libre.
- **Flask:** licencia BSD — uso libre.
- **React:** licencia MIT — uso libre.
- **PostgreSQL:** licencia PostgreSQL (similar a BSD) — uso libre.
- **Docker:** licencia Apache 2.0 — uso libre.
- **Ultralytics YOLOv8:** licencia AGPL-3.0 para uso no comercial — compatible con el uso académico de este proyecto.
- **OpenCV:** licencia Apache 2.0 — uso libre.
- **BORIS:** licencia GNU GPL v3 — uso libre para investigación [32].

Los videos que usa el sistema son propiedad del grupo de investigación de la ENMyH-IPN y su uso en este proyecto está respaldado por la colaboración formal entre los directores del TT y el laboratorio. No se publican ni se comparten fuera del proyecto.

El equipo de desarrollo no interviene en ningún experimento con animales. Eso lo hace exclusivamente el laboratorio, que ya tiene los permisos institucionales y éticos necesarios. El sistema solo analiza las grabaciones que resultan de esos experimentos.

4.5. Análisis de sostenibilidad

Más allá de que el proyecto funcione técnicamente, vale la pena preguntarse qué pasa con él a largo plazo.

4.5.1. Sostenibilidad ambiental

Reducir el análisis manual significa que los investigadores no tienen que ir al laboratorio solo para sentarse a cronometrar videos. Como el sistema es web y se puede usar desde cualquier lugar, el desplazamiento para esas tareas desaparece.

El costo energético de procesar video en una computadora es marginal comparado con las horas de trabajo que reemplaza. Y la política de borrar los videos a los 30 días evita que el disco crezca sin control.

4.5.2. Sostenibilidad social e institucional

El sistema está hecho para el laboratorio de la ENMyH, pero está diseñado de forma modular para que pueda adaptarse a otros paradigmas conductuales (laberinto en cruz elevada, campo abierto, etc.) solo reconfigurando el módulo de clasificación. Eso abre la puerta a que otros grupos del IPN puedan usarlo sin tener que desarrollarlo desde cero.

Al estandarizar el análisis y quitarle variabilidad humana al proceso, el sistema también contribuye a que los resultados del laboratorio sean más reproducibles, lo que tiene un impacto directo en la calidad de la investigación [4].

4.5.3. Mantenibilidad del sistema

El sistema se entrega como prototipo funcional. Para mantenerlo después, hace falta alguien que sepa lo básico de Python y Docker; no se requiere ser experto. La documentación técnica de este documento y el `README` del repositorio están pensados para que cualquier persona que retome el proyecto pueda entender cómo funciona sin necesitar al equipo original.

Las herramientas principales (YOLOv8, Flask, React, PostgreSQL) siguen en desarrollo activo y tienen comunidades grandes, así que no hay riesgo de que queden obsoletas en el horizonte de tiempo que se considera para este prototipo.

Capítulo 5

Desarrollo

5.1. Entorno de desarrollo

5.2. Implementación del módulo de seguimiento (tracker.py)

5.2.1. Representación de imágenes y video

Para el módulo de seguimiento, un video se trata como una secuencia de matrices de números. Cada cuadro (*frame*) es una imagen BGR de tres canales (azul, verde, rojo), que es la convención que usa OpenCV. Un video de 5 minutos a 30 FPS equivale a procesar 9,000 cuadros de forma secuencial.

5.2.2. Modelo de fondo y sustracción

El primer paso del pipeline es construir un modelo del fondo: una imagen de referencia del cilindro sin la rata. Se calcula como la mediana píxel a píxel de los primeros 60 cuadros del video. Usar la mediana en vez del promedio evita que una rata inmóvil al inicio "contamine" el modelo. Una vez que se tiene el fondo, en cada cuadro se calcula la diferencia absoluta contra esa referencia; los píxeles que cambiaron significativamente corresponden al primer plano (la rata).

Este enfoque es más simple que métodos adaptativos como el modelo de mezcla gaussiana (GMM) [27], pero suficiente para el caso de uso: la cámara es fija y el fondo no cambia durante el experimento.

5.2.3. Umbralización y limpieza morfológica

La imagen de diferencias se convierte a binario mediante el método de Otsu [33], que calcula automáticamente el umbral óptimo a partir del histograma de intensidades. Después se aplica una apertura morfológica (erosión seguida de dilatación) para eliminar ruido pequeño y rellenar huecos en la región detectada.

5.2.4. Detección de contornos y selección de candidatos

Sobre la imagen binaria limpia se extraen los contornos con `findContours` de OpenCV [34]. De cada contorno se obtiene el área, el rectángulo envolvente (*bounding box*) y el centroide. Con esas propiedades se filtra cuál de los candidatos detectados corresponde a la rata y cuál es ruido, usando restricciones de área mínima y máxima calibradas para el tamaño esperado del animal dentro de cada cilindro.

Para verificar consistencia entre cuadros consecutivos se usa la métrica IoU (*Intersection over Union*): la relación entre el área de intersección y el área de unión de dos rectángulos. Un IoU cercano a 1 indica que la detección actual coincide con la del cuadro anterior; si es demasiado bajo se activa el respaldo del tracker de correlación.

5.2.5. Estimación de la línea de agua

La superficie del agua puede generar reflejos que hacen que el animal aparezca duplicado desde la vista superior. Para mitigar ese efecto, el sistema estima en cada cuadro la posición vertical de la línea del agua: se analiza el perfil de intensidad promedio por filas dentro de cada ROI y se identifica la fila con el cambio más brusco. Como esa estimación oscila por ruido entre cuadros, se estabiliza con un promedio móvil exponencial (EMA):

$$\hat{y}_t = \alpha \cdot \tilde{y}_t + (1 - \alpha) \cdot \hat{y}_{t-1} \tag{5.1}$$

donde $\tilde{y}_t$ es la estimación del cuadro actual, $\hat{y}_{t-1}$ es el valor suavizado anterior, y α controla la velocidad de respuesta. Esta lógica está encapsulada en la clase `WaterlineSmoother`. Su efectividad depende de las condiciones del video y se evaluará durante las pruebas de validación.

5.3. Implementación del clasificador de conducta

5.4. Implementación del backend (Flask)

5.5. Implementación del frontend (React + Vite)

5.6. Integración y despliegue con Docker Compose

Bibliografía

[1] World Health Organization, *Trastorno depresivo (depresión)*, 2025. dirección: https://www.who.int/es/news-room/fact-sheets/detail/depression (visitado 01-02-2026).

[2] R. D. Porsolt, A. Bertin y M. Jalfre, «Behavioral despair in mice: A primary screening test for antidepressants,» *Archives Internationales de Pharmacodynamie et de Therapie*, vol. 229, n.º 2, págs. 327-336, 1977.

[3] D. A. Slattery y J. F. Cryan, «Using the rat forced swim test to assess antidepressant-like activity in rodents,» *Nature Protocols*, vol. 7, n.º 6, págs. 1009-1014, 2012. DOI: 10.1038/nprot.2012.044.

[4] A. Armario, «The forced swim test: Historical, conceptual and methodological considerations and its relationship with individual behavioral traits,» *Neuroscience and Biobehavioral Reviews*, vol. 128, págs. 74-86, 2021. DOI: 10.1016/j.neubiorev.2021.06.014.

[5] Noldus Information Technology, *EthoVision XT | Video tracking software*, n.d. dirección: https://noldus.com/ethovision-xt (visitado 01-02-2026).

[6] Z. T. Pennington, Z. Dong, Y. Feng y col., «ezTrack: An open-source video analysis pipeline for the investigation of animal behavior,» *Scientific Reports*, vol. 9, n.º 1, pág. 19 979, 2019. DOI: 10.1038/s41598-019-56408-9.

[7] A. Della Valle, S. De Carlo, G. Sonsini, M. Ubaldi y R. Ciccocioppo, «Machine learning-based model for behavioural analysis in rodents applied to the forced swim test,» *Scientific Reports*, vol. 15, 2025. DOI: 10.1038/s41598-025-05712-8.

[8] A. Nandi, G. Virmani, A. Barve y S. Marathe, «DBscorer: An Open-Source Software for Automated Accurate Analysis of Rodent Behavior in Forced Swim Test and Tail Suspension Test,» *eNeuro*, vol. 8, n.º 6, 2021. DOI: 10.1523/ENEURO.0305-21.2021.

[9] X. Meng, Y. Xia, M. Liu y col., «A deep-learning-based threshold-free method for automated analysis of rodent behavior in the forced swim test and tail suspension test,» *Journal of Neuroscience Methods*, vol. 409, pág. 110 212, 2024. DOI: 10.1016/j.jneumeth.2024.110212.

[10] A. Mathis, P. Mamidanna, K. M. Cury y col., «DeepLabCut: Markerless pose estimation of user-defined body parts with deep learning,» *Nature Neuroscience*, vol. 21, n.º 9, págs. 1281-1289, 2018. DOI: 10.1038/s41593-018-0209-y.

[11] J. Lauer, M. Zhou, S. Ye y col., «Multi-animal pose estimation, identification and tracking with DeepLabCut,» *Nature Methods*, vol. 19, n.º 4, págs. 496-504, 2022. DOI: 10.1038/s41592-022-01443-0.

[12] O. Sturman, L. von Ziegler, C. Schlaeppi y col., «Deep learning-based behavioral analysis enables high precision rodent tracking and is capable of outperforming commercial solutions,» *Neuropsychopharmacology*, vol. 45, págs. 1137-1144, 2020. DOI: 10.1038/s41386-020-0776-y.

[13] Y.-H. Chan, J. A. Klarevas-Irby y D. R. Farine, «YOLO-Behaviour: A simple, flexible framework to automatically quantify animal behaviours from videos,» *Methods in Ecology and Evolution*, 2025. DOI: 10.1111/2041-210X.14502.

[14] J. H. Stadler, H. Luckow y C. Goeke, «Automatic object detection for behavioural research using YOLOv8,» *Behavior Research Methods*, vol. 56, n.º 7, págs. 7307-7330, 2024. DOI: 10.3758/s13428-024-02420-5.

[15] J. F. Cryan, C. Mombereau y A. Vassout, «The tail suspension test as a model for assessing antidepressant activity,» *Neuroscience and Biobehavioral Reviews*, vol. 29, págs. 571-625, 2005. DOI: 10.1016/j.neubiorev.2005.03.009.

[16] D. P. Kingma y J. Ba, «Adam: A Method for Stochastic Optimization,» en *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015. dirección: https://arxiv.org/abs/1412.6980.

[17] Y. LeCun, L. Bottou, Y. Bengio y P. Haffner, «Gradient-based learning applied to document recognition,» *Proceedings of the IEEE*, vol. 86, n.º 11, págs. 2278-2324, 1998. DOI: 10.1109/5.726791.

[18] S. J. Pan y Q. Yang, «A survey on transfer learning,» *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, n.º 10, págs. 1345-1359, 2010. DOI: 10.1109/TKDE.2009.191.

[19] K. He, X. Zhang, S. Ren y J. Sun, «Deep Residual Learning for Image Recognition,» en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, págs. 770-778. DOI: 10.1109/CVPR.2016.90.

[20] A. Lukezic, T. Vojir, L. Cehovin, J. Matas y M. Kristan, «Discriminative Correlation Filter with Channel and Spatial Reliability,» en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, págs. 4847-4856. DOI: 10.1109/CVPR.2017.515.

[21] J. F. Henriques, R. Caseiro, P. Martins y J. Batista, «High-Speed Tracking with Kernelized Correlation Filters,» *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, n.º 3, págs. 583-596, 2015. DOI: 10.1109/TPAMI.2014.2345390.

[22] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8.ª ed. McGraw-Hill Education, 2014.

[23] I. Sommerville, *Software Engineering*, 10.ª ed. Pearson, 2016.

[24] G. Bradski y A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.

[25] G. Jocher, A. Chaurasia y J. Qiu, *Ultralytics YOLO (Version 8.0.0)*, 2023. dirección: https://github.com/ultralytics/ultralytics (visitado 01-02-2026).

[26] M. Piccardi, «Background subtraction techniques: a review,» en *Proceedings of the IEEE International Conference on Systems, Man and Cybernetics*, 2004, págs. 3099-3104. DOI: 10.1109/ICSMC.2004.1400815.

[27] Z. Zivkovic, «Improved adaptive Gaussian mixture model for background subtraction,» en *Proceedings of the 17th International Conference on Pattern Recognition (ICPR)*, vol. 2, 2004, págs. 28-31. DOI: 10.1109/ICPR.2004.1333992.

[28] M. Abadi, P. Barham, J. Chen y col., «TensorFlow: A System for Large-Scale Machine Learning,» en *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, págs. 265-283. dirección: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

[29] F. Pedregosa, G. Varoquaux, A. Gramfort y col., «Scikit-learn: Machine Learning in Python,» *Journal of Machine Learning Research*, vol. 12, págs. 2825-2830, 2011.

[30] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2.ª ed. O'Reilly Media, 2018.

[31] D. Merkel, «Docker: Lightweight Linux Containers for Consistent Development and Deployment,» *Linux Journal*, vol. 2014, n.º 239, pág. 2, 2014.

[32] O. Friard y M. Gamba, «BORIS: a free, versatile open-source event-logging software for video/audio coding and live observations,» *Methods in Ecology and Evolution*, vol. 7, n.º 11, págs. 1325-1330, 2016. DOI: 10.1111/2041-210X.12584.

[33] N. Otsu, «A threshold selection method from gray-level histograms,» *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, n.º 1, págs. 62-66, 1979. DOI: 10.1109/TSMC.1979.4310076.

[34] S. Suzuki y K. Abe, «Topological structural analysis of digitized binary images by border following,» *Computer Vision, Graphics, and Image Processing*, vol. 30, n.º 1, págs. 32-46, 1985. DOI: 10.1016/0734-189X(85)90016-7.